Aula: Arrays em JavaScript

Elaborar Projetos de Aplicações para Web · Trilha de JavaScript

Objetivos da aula

Ao final desta aula, o aluno será capaz de:

- Explicar o que é um array e quando usá-lo
- Criar, ler, alterar e percorrer arrays
- Usar os principais métodos: push, pop, splice, slice, map, filter, reduce, find, sort e outros
- Trabalhar com arrays de objetos (o formato mais comum em aplicações web)
- Aplicar arrays em situações reais: listas na tela, carrinho de compras, dados salvos no localStorage

Pré-requisitos: variáveis, tipos de dados, funções e estruturas de repetição (for).

1. Definição

Um **array** é uma estrutura de dados que guarda **vários valores em uma única variável**, em uma **ordem definida**. Cada valor é chamado de **elemento** e ocupa uma posição numerada chamada **índice**, que **começa em 0**.

Analogia: pense em uma fila de caixas de correio numeradas. O prédio (a variável) é um só, mas cada caixa (o índice) guarda uma correspondência (o valor). Para pegar algo, basta dizer o número da caixa.

```
const sabores = ['Calabresa', 'Mussarela', 'Portuguesa'];
//                índice 0     índice 1     índice 2
```

Características importantes em JavaScript:

- Arrays podem guardar **tipos misturados** (texto, número, objeto, outro array)
- O tamanho é **dinâmico**: cresce e diminui conforme necessário
- Arrays são **objetos**: `typeof []` retorna `'object'`. Para checar de verdade, use `Array.isArray()`
- Podem ser declarados com `const` e ainda assim ter o conteúdo alterado (o que não muda é a referência)

```
console.log(Array.isArray(sabores)); // true
console.log(typeof sabores);         // 'object'
```

2. Criando arrays

```
// Forma literal (a mais usada)
const frutas = ['maçã', 'banana', 'uva'];

// Array vazio
const vazio = [];

// Tipos misturados (possível, mas use com critério)
const misto = ['Ana', 25, true, null];

// Construtor Array
const cinco = new Array(5);        // 5 posições vazias

// Array.of e Array.from
const numeros = Array.of(1, 2, 3);          // [1, 2, 3]
const letras = Array.from('abc');           // ['a', 'b', 'c']
const dobro = Array.from({ length: 3 }, (_, i) => i * 2); // [0, 2, 4]
```

Dica: prefira sempre a forma literal `[]`. O `new Array(5)` cria um array de um item vazio com tamanho 5, e `new Array(5)` é diferente de `new Array('5')`, o que gera confusão.

3. Acessando e alterando elementos

```
const cores = ['azul', 'verde', 'vermelho'];

console.log(cores[0]);               // 'azul'
console.log(cores[2]);               // 'vermelho'
console.log(cores[10]);              // undefined (não existe)
console.log(cores.length);           // 3
console.log(cores[cores.length - 1]); // 'vermelho' (último)
console.log(cores.at(-1));           // 'vermelho' (forma moderna)

cores[1] = 'amarelo';                // altera
console.log(cores);                  // ['azul', 'amarelo', 'vermelho']
```

- `length` é sempre o **último índice + 1**
- Acessar um índice que não existe **não gera erro**: retorna `undefined`
- `at()` aceita índices negativos, contando a partir do fim

4. Métodos para adicionar e remover elementos

Estes métodos **alteram o array original** (são chamados de *mutadores*).

Método	O que faz	Retorna
`push(x)`	Adiciona no **final**	novo tamanho
`pop()`	Remove do **final**	o elemento removido
`unshift(x)`	Adiciona no **início**	novo tamanho
`shift()`	Remove do **início**	o elemento removido
`splice(i, n, ...novos)`	Remove e/ou insere a partir do índice i	array com os removidos

```
const fila = ['Ana', 'Bruno'];

fila.push('Carla');        // ['Ana', 'Bruno', 'Carla']
fila.unshift('Zeca');      // ['Zeca', 'Ana', 'Bruno', 'Carla']
fila.pop();                // remove 'Carla'
fila.shift();              // remove 'Zeca'
console.log(fila);         // ['Ana', 'Bruno']
```

Analogia: `push` e `pop` funcionam como uma pilha de pratos (entra e sai pelo topo). `push` e `shift` funcionam como uma fila de banco (entra no fim, sai pelo começo).

splice: o canivete suíço

```
const letras = ['a', 'b', 'c', 'd', 'e'];

letras.splice(2, 1);            // remove 1 item a partir do índice 2 ->
['a','b','d','e']
letras.splice(1, 0, 'X');       // insere 'X' no índice 1 ->
['a','X','b','d','e']
letras.splice(3, 1, 'Y', 'Z');  // troca 'd' por 'Y' e 'Z' ->
['a','X','b','Y','Z','e']
```

5. Métodos para buscar, copiar e transformar em texto

Estes métodos **não alteram** o array original.

```
const nums = [10, 20, 30, 20, 40];

nums.indexOf(20);       // 1 (primeira posição) | -1 se não achar
nums.lastIndexOf(20);   // 3
nums.includes(30);      // true

nums.slice(1, 3);       // [20, 30] (do índice 1 até antes do 3)
nums.slice(-2);         // [20, 40] (últimos dois)
nums.slice();           // cópia do array inteiro

[1, 2].concat([3, 4]);  // [1, 2, 3, 4]

['HTML', 'CSS', 'JS'].join(' | ');   // 'HTML | CSS | JS'
'a,b,c'.split(',');                   // ['a', 'b', 'c'] (método de string)
```

Atenção: `slice` copia (não altera); `splice` corta (altera). Uma forma de lembrar: o **p** de spli**ce** de "pedaço removido do original".

6. Percorrendo arrays

```
const itens = ['pão', 'leite', 'ovos'];

// 1) for clássico: controle total do índice
for (let i = 0; i < itens.length; i++) {
  console.log(i, itens[i]);
}

// 2) for...of: simples, quando só precisa do valor
for (const item of itens) {
  console.log(item);
}

// 3) forEach: executa uma função para cada elemento
itens.forEach((item, indice) => {
  console.log(`${indice + 1}. ${item}`);
});

// 4) entries: índice e valor juntos
for (const [i, item] of itens.entries()) {
  console.log(i, item);
}
```

Evite `for...in` em arrays: ele percorre as chaves (como texto) e pode incluir propriedades inesperadas.

7. Métodos de ordem superior

São métodos que **recebem uma função** como argumento. São a base da programação moderna em JavaScript e aparecem em praticamente todo projeto web.

map: transforma cada elemento (retorna um novo array do mesmo tamanho)

```
const precos = [10, 20, 30];
const comDesconto = precos.map(p => p * 0.9);   // [9, 18, 27]
```

filter: mantém só os que passam no teste

```
const idades = [12, 18, 25, 16, 40];
const maiores = idades.filter(i => i >= 18);    // [18, 25, 40]
```

reduce: reduz o array a um único valor

```
const total = [10, 20, 30].reduce((acumulador, atual) => acumulador + atual,
0);
console.log(total); // 60
```

`reduce` recebe a função e o **valor inicial** do acumulador (o `0` acima). Sempre informe o valor inicial.

find e findIndex: acham o primeiro que passa no teste

```
const usuarios = [
  { id: 1, nome: 'Ana' },
  { id: 2, nome: 'Bruno' }
];

usuarios.find(u => u.id === 2);        // { id: 2, nome: 'Bruno' }
usuarios.findIndex(u => u.id === 2);   // 1
usuarios.find(u => u.id === 9);        // undefined
```

some e every: testes verdadeiro/falso

```
const notas = [7, 8, 5, 9];
notas.some(n => n < 6);    // true  (existe alguma abaixo de 6?)
notas.every(n => n >= 6);  // false (todas são 6 ou mais?)
```

Encadeando métodos

```
const notas = [4, 7, 9, 5, 10];
const mediaAprovados = notas
  .filter(n => n >= 6)
  .reduce((soma, n, _, arr) => soma + n / arr.length, 0);
```

Método	Retorna	Use quando
`forEach`	nada	só executar uma ação (ex.: imprimir)
`map`	novo array	transformar cada item
`filter`	novo array	selecionar itens
`reduce`	um valor	somar, contar, agrupar
`find`	um item ou undefined	achar o primeiro
`some` / `every`	true ou false	testar condição

8. Ordenação e inversão

```
const nomes = ['Carla', 'Ana', 'Bruno'];
nomes.sort();            // ['Ana', 'Bruno', 'Carla']
nomes.reverse();         // ['Carla', 'Bruno', 'Ana']
```

Cuidado com números: `sort()` sem função compara como **texto**.

```
[10, 9, 100, 1].sort();                 // [1, 10, 100, 9]  (errado!)
[10, 9, 100, 1].sort((a, b) => a - b);  // [1, 9, 10, 100]  (crescente)
[10, 9, 100, 1].sort((a, b) => b - a);  // [100, 10, 9, 1]  (decrescente)
```

Ordenando objetos por uma propriedade:

```
produtos.sort((a, b) => a.preco - b.preco);        // por preço
produtos.sort((a, b) => a.nome.localeCompare(b.nome)); // por nome (acentos
ok)
```

`sort` e `reverse` **alteram o original**. Para preservar, copie antes: `[...nomes].sort()`.

9. Spread e desestruturação

```
// Copiar e juntar com spread
const a = [1, 2];
const b = [3, 4];
const copia = [...a];          // [1, 2]
const juntos = [...a, ...b];   // [1, 2, 3, 4]
const novo = [0, ...a, 99];    // [0, 1, 2, 99]

// Desestruturação
const [primeiro, segundo] = ['Ana', 'Bruno', 'Carla'];
const [lider, ...resto] = ['Ana', 'Bruno', 'Carla'];
// lider = 'Ana'; resto = ['Bruno', 'Carla']

// Trocar valores sem variável auxiliar
let x = 1, y = 2;
[x, y] = [y, x];
```

Copiar não é atribuir: `const b = a` **não** cria uma cópia, apenas outro nome para o mesmo array. Alterar `b` altera `a`.

```
const a = [1, 2];
const b = a;        // mesma referência
b.push(3);
console.log(a);     // [1, 2, 3]  (surpresa!)
```

10. Arrays de objetos e arrays multidimensionais

Array de objetos (o formato dos dados reais)

APIs, bancos de dados e o localStorage trabalham, quase sempre, com listas de objetos.

```
const cardapio = [
  { id: 1, nome: 'Calabresa', preco: 42, vegetariana: false },
  { id: 2, nome: 'Margherita', preco: 38, vegetariana: true },
  { id: 3, nome: 'Quatro Queijos', preco: 48, vegetariana: true }
];

cardapio.filter(p => p.vegetariana);          // só as vegetarianas
cardapio.map(p => p.nome);                    // ['Calabresa', 'Margherita',
'Quatro Queijos']
cardapio.reduce((s, p) => s + p.preco, 0);   // 128
cardapio.find(p => p.id === 3).nome;          // 'Quatro Queijos'
```

Array multidimensional (matriz)

```
const tabela = [
  ['Ana', 8, 9],
  ['Bruno', 6, 7]
];

console.log(tabela[1][0]);   // 'Bruno' (linha 1, coluna 0)

for (const linha of tabela) {
  for (const celula of linha) {
    console.log(celula);
  }
}
```

Para "achatar" arrays aninhados: `[1, [2, [3]]].flat(Infinity)` retorna `[1, 2, 3]`.

11. Usos principais em aplicações web

1) Renderizar listas no HTML

```
const lista = document.querySelector('#lista');
lista.innerHTML = cardapio
  .map(p => `<li>${p.nome} - R$ ${p.preco}</li>`)
  .join('');
```

2) Carrinho de compras: adicionar (`push`), remover (`filter` ou `splice`), somar total (`reduce`).

```
let carrinho = [];
carrinho.push(cardapio[0]);
carrinho = carrinho.filter(item => item.id !== 1);    // remove pelo id
```

3) Salvar e recuperar no localStorage (que só guarda texto, por isso o JSON):

```
localStorage.setItem('carrinho', JSON.stringify(carrinho));
const salvo = JSON.parse(localStorage.getItem('carrinho')) || [];
```

4) Filtros e buscas: campo de pesquisa usando `filter` com `includes`.

```
const busca = 'queijo';
const achados = cardapio.filter(p => p.nome.toLowerCase().includes(busca));
```

5) Validar formulários: guardar mensagens de erro em um array e exibi-las de uma vez.

6) Consumir APIs: o resultado de um `fetch` costuma ser um array de objetos para percorrer e exibir.

7) Elementos do DOM: `querySelectorAll` retorna uma NodeList. Para usar `map` / `filter`, converta: `Array.from(lista)` ou `[...lista]`.

12. Boas práticas e erros comuns

Boas práticas

- Use `const` para arrays e nomes no plural (`alunos`, `produtos`)
- Prefira `map`, `filter` e `reduce` a laços com variáveis auxiliares: o código fica mais curto e legível
- Evite alterar o array original quando não for necessário: copie com `[...arr]` ou `slice()`
- Verifique com `Array.isArray()` quando o valor puder vir de fora (API, formulário)
- Guarde elementos do mesmo tipo/formato no mesmo array

Erros comuns

Erro	Consequência	Correção
Achar que o índice começa em 1	Pega o item errado ou `undefined`	Lembrar: primeiro é `[0]`, último é `[length - 1]`
`for (i = 0; i <= arr.length; i++)`	Passa do fim, gera `undefined`	Usar `<` em vez de `<=`
`sort()` com números	Ordem errada ("100" antes de "9")	Passar `(a, b) => a - b`
`const b = a` achando que copia	Alterar `b` altera `a`	Usar `[...a]`
Usar `map` só para imprimir	Cria array desnecessário	Usar `forEach`
`reduce` sem valor inicial	Erro em array vazio	Sempre informar o valor inicial
Esquecer `return` em chaves no `map` / `filter`	Retorna `undefined`	Usar arrow sem chaves ou colocar `return`
Comparar arrays com `==`	Sempre `false` (compara referência)	Comparar item a item ou `JSON.stringify`

13. Atividades

Bloco 1: Fixação (básico)

Atividade 1. Crie um array com 5 frutas. Mostre no console a primeira, a última e a quantidade de itens.

Atividade 2. Dado `const numeros = [3, 8, 1, 9, 4];`, faça:

- Adicione o número 7 no final e o número 0 no início
- Remova o último elemento
- Mostre o array final

Atividade 3. Dado `const nomes = ['Ana', 'Bruno', 'Carla', 'Diego'];`, escreva um programa que:

- Percorra o array com `for...of` e mostre `"Olá, Ana!"` para cada nome
- Verifique se `'Carla'` está no array e mostre `true` ou `false`
- Descubra em qual posição está `'Diego'`

Bloco 2: Aplicação (intermediário)

Atividade 4. Dado `const notas = [7.5, 4, 9, 6, 3.5, 10];`:

- Calcule a **média** com `reduce`
- Crie um novo array só com as notas **aprovadas** (6 ou mais) usando `filter`
- Crie um array com as notas **arredondadas** usando `map` e `Math.round`
- Descubra a maior nota (dica: `Math.max(...notas)`)

Atividade 5. Dado o array abaixo, resolva os itens:

```
const alunos = [
  { nome: 'Ana', nota: 8.5, turma: 'A' },
  { nome: 'Bruno', nota: 5.0, turma: 'B' },
  { nome: 'Carla', nota: 9.2, turma: 'A' },
  { nome: 'Diego', nota: 6.5, turma: 'B' }
];
```

- a) Liste só os nomes dos alunos
- b) Liste os alunos da turma A
- c) Encontre o aluno chamado Carla
- d) Ordene os alunos da maior para a menor nota (sem alterar o array original)
- e) Verifique se existe algum aluno reprovado (nota abaixo de 6) e se todos têm nota acima de 5

Atividade 6. Escreva uma função `removerDuplicados(arr)` que receba um array e devolva outro sem repetições. Teste com `[1, 2, 2, 3, 3, 3, 4]`. (Dica: `filter` com `indexOf`, ou `[...new Set(arr)]`.)

Bloco 3: Desafio

Atividade 7. Contador de palavras. Dada a frase `'o rato roeu a roupa do rei de roma'`, use `split`, `reduce` e um objeto para contar quantas vezes cada palavra aparece.

Atividade 8. Matriz. Dada a matriz abaixo, calcule a soma de cada linha e a soma total:

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

Bloco 4: Mini-projeto integrador (conectado ao projeto da pizzaria)

Atividade 9. Carrinho da pizzaria. Usando o array `cardapio` da seção 10, implemente:

1. Uma função `adicionar(id)` que localiza a pizza com `find` e a coloca no array `carrinho`
2. Uma função `remover(id)` que tira a pizza do carrinho com `filter`
3. Uma função `total()` que retorna o valor do pedido com `reduce`
4. Uma função `renderizar()` que mostre os itens do carrinho em uma `<ul>` do HTML usando `map` e `join`
5. Salvar o carrinho no `localStorage` a cada alteração e recuperá-lo ao carregar a página
6. **Extra:** um campo de busca que filtre o cardápio enquanto o usuário digita

Critérios de avaliação: uso correto de `map`, `filter`, `find` e `reduce`; não alterar o array `cardapio` original; código organizado em funções; dados persistindo após recarregar a página.

14. Gabarito

```
// Atividade 2
const numeros = [3, 8, 1, 9, 4];
numeros.push(7);
numeros.unshift(0);
numeros.pop();
console.log(numeros); // [0, 3, 8, 1, 9, 4]
```

```
// Atividade 3
const nomes = ['Ana', 'Bruno', 'Carla', 'Diego'];
for (const nome of nomes) console.log(`Olá, ${nome}!`);
console.log(nomes.includes('Carla'));  // true
console.log(nomes.indexOf('Diego'));   // 3

// Atividade 4
const notas = [7.5, 4, 9, 6, 3.5, 10];
const media = notas.reduce((s, n) => s + n, 0) / notas.length; // 6.67
const aprovadas = notas.filter(n => n >= 6);                   // [7.5, 9,
6, 10]
const arredondadas = notas.map(n => Math.round(n));            // [8, 4, 9,
6, 4, 10]
const maior = Math.max(...notas);                              // 10

// Atividade 5
alunos.map(a => a.nome);                          // a
alunos.filter(a => a.turma === 'A');              // b
alunos.find(a => a.nome === 'Carla');             // c
[...alunos].sort((a, b) => b.nota - a.nota);      // d
alunos.some(a => a.nota < 6);                     // e: true
alunos.every(a => a.nota > 5);                    // e: false (Bruno tem 5.0)

// Atividade 6
const removerDuplicados = arr => [...new Set(arr)];
console.log(removerDuplicados([1, 2, 2, 3, 3, 3, 4])); // [1, 2, 3, 4]

// Atividade 7
const frase = 'o rato roeu a roupa do rei de roma';
const contagem = frase.split(' ').reduce((acc, palavra) => {
  acc[palavra] = (acc[palavra] || 0) + 1;
  return acc;
}, {});
// { o: 1, rato: 1, roeu: 1, a: 1, roupa: 1, do: 1, rei: 1, de: 1, roma: 1 }

// Atividade 8
const somas = matriz.map(linha => linha.reduce((s, n) => s + n, 0)); // [6,
15, 24]
const totalGeral = somas.reduce((s, n) => s + n, 0);               // 45

// Atividade 9 (esqueleto)
let carrinho = JSON.parse(localStorage.getItem('carrinho')) || [];

function salvar() {
  localStorage.setItem('carrinho', JSON.stringify(carrinho));
}
```

```
function adicionar(id) {
  const pizza = cardapio.find(p => p.id === id);
  if (pizza) { carrinho.push(pizza); salvar(); renderizar(); }
}
function remover(id) {
  carrinho = carrinho.filter(p => p.id !== id);
  salvar(); renderizar();
}
function total() {
  return carrinho.reduce((soma, p) => soma + p.preco, 0);
}
function renderizar() {
  document.querySelector('#carrinho').innerHTML = carrinho
    .map(p => `<li>${p.nome} - R$ ${p.preco}</li>`)
    .join('');
}
renderizar();
```

15. Resumo rápido

Preciso de...	Use
Adicionar no fim / início	`push` / `unshift`
Remover do fim / início	`pop` / `shift`
Inserir ou remover no meio	`splice`
Copiar um trecho	`slice`
Saber se existe	`includes`, `some`
Achar a posição	`indexOf`, `findIndex`
Achar um item	`find`
Transformar todos	`map`
Selecionar alguns	`filter`
Somar / acumular	`reduce`
Ordenar	`sort` (com função para números)
Juntar / copiar	`concat`, spread `[...a, ...b]`
Virar texto	`join`

Para casa: refazer a Atividade 9 acrescentando um campo de quantidade por item (dica: `findIndex` para saber se a pizza já está no carrinho).